Design Journal: Weighted Decision-Diagram Backend

Implementation milestones, validation, and benchmark observations

Project and responsibility

Contribution: Weighted decision-diagram (DD) backend for the gate-based quantum circuit simulator.
Main implementation: Python backend interface and C++ DD core; benchmark integration through the project harness.
Goal: Implement a simulator backend that represents quantum states as a weighted graph and evaluate its correctness and performance on several circuit families.

Implementation journal

1. State representation and canonicalization. The DD core represents an edge using a complex weight and a node identifier. Each internal node stores low and high outgoing edges at a qubit level. During node construction, edge weights are normalized and canonical nodes are interned in a unique table. This enables equivalent subgraphs to be shared instead of storing every amplitude independently.

2. Gate application and simulator interface. Recursive DD operations were implemented to apply gates to the represented state. The Python interface connects the simulator API to the C++ implementation. The backend supports the project operations exercised by the harness, including single- and two-qubit gates, controlled operations, measurement and sampling, amplitude access, and Pauli expectation values.

3. Memory management and operation caches. Operation caches are used to reuse intermediate results during recursive computations. A compaction routine removes nodes that are no longer reachable from the current root and remaps the retained graph. Compaction is triggered using node-count and gate-count thresholds to control graph growth. The cache-clearing and compaction logic are part of the implementation's resource-management strategy.

4. Validation and debugging. The backend was tested with the project's correctness harness and DD-specific unit tests. Recorded checks covered GHZ, QFT, and Bernstein–Vazirani circuits at multiple sizes, as well as controlled gates, swaps, controlled-U, expectations, and measurement collapse. In the recorded harness run, all 15 GHZ/QFT/BV cases passed, with fidelity reported as 1.000000 and the expected BV bitstrings recovered. These results describe the recorded local run; the final integrated submission should still be checked in the target environment.

Benchmark experiments

The benchmark suite includes six workloads: GHZ, QFT, Bernstein–Vazirani (BV), one-dimensional QAOA, near-Clifford, and random-universal circuits. The supplied CSVs record per-size measurements, and the corresponding plots show runtime scaling. The harness reports median runtime over three runs and records its prescribed memory metric on the first run; that measurement procedure was left unchanged.

Workload	Largest n	DD median time
GHZ	20	0.00469 s
QFT	20	0.01119 s
Bernstein–Vazirani	20	0.00569 s
1D QAOA	14	2.29166 s
Near-Clifford	20	2.87772 s
Random-universal	14	54.01464 s

Observations and design interpretation

The tested GHZ, QFT, and BV circuits complete in milliseconds at 20 qubits. The near-Clifford workload takes about 2.88 seconds at 20 qubits, while QAOA takes about 2.29 seconds at 14 qubits. Random-universal is the most expensive tested case at about 54.01 seconds for 14 qubits.

This difference is consistent with the DD representation benefiting when a circuit produces repeated or equivalent subgraphs. Circuits that provide less opportunity for sharing can cause the graph and recursive computation cost to grow substantially. These measurements are workload-specific and do not establish a universal complexity guarantee.

Aer timing values are available in the CSVs for GHZ, QFT, BV, and QAOA, but not for near-Clifford or random-universal. Comparisons should be treated as indicative because randomized circuit instances or parameters may differ between backend runs. The reported memory metric is the one generated by the supplied harness and should not be interpreted as a complete measurement of native C++ memory.

Current status and remaining work

- **Implemented:** weighted DD state representation, canonical node sharing, recursive gate operations, Python/C++ interface, and graph compaction.
- **Recorded validation:** DD unit tests and the 15-case GHZ/QFT/BV harness run passed in the recorded local environment.
- **Recorded experiments:** six benchmark CSVs and six runtime plots are available in the project.
- **Before submission:** rerun the correctness tests from the final project directory, verify that the required source files and plots are included, and ensure the report matches the submitted CSVs.

Artifacts: `results/dd_*.csv` and `results_dd_*.png`.
Reference context: Decision diagrams exploit repeated structure in quantum states; their practical performance depends on the graph size generated by the circuit.